

NutriTrack — full-stack curriculum

Food calorie & macro tracker built phase by phase, where every concept compounds

Stack	Python · FastAPI · Flask · PostgreSQL · React · Docker · Railway
Output	Live deployed app accessible by anyone on the internet
Design principle	Every concept introduced in any phase is reused in all subsequent phases
Estimated time	15–25 days at 10 hours/day depending on prior experience

Phase 1 — Python core foundations		complete
Deliverable: <code>nutritrack/core/</code> — domain models + utility layer		Tools: Python · mypy · dataclasses · itertools · logging
1.1	Project setup & type hints	[done]
1.2	Dataclasses & domain models	[done]
1.3	Error handling & custom exceptions	[done]
1.4	Decorators & closures	[done]
1.5	Generators & lazy pipelines	[done]
1.6	Collections & MacroAggregator	[done]
1.7	Logging & structured output	[done]
Phase 2 — OOP & data modeling		complete
Deliverable: <code>nutritrack/db/</code> — full persistence layer with migrations		Tools: PostgreSQL · SQLAlchemy · Alembic · Pydantic
2.1	PostgreSQL setup & SQLAlchemy base	[done]
2.2	ORM models	[done]
2.3	Alembic migrations	[done]
2.4	Repository pattern — CRUD operations	[done]

2.5	Pydantic schemas	[current]
2.6	Database seeding	[]
Phase 3 — FastAPI backend		upcoming
Deliverable: nutritrack/api/ — REST API with auth		Tools: FastAPI · JWT · async/await · middleware
3.1	FastAPI setup & app structure	[]
3.2	Auth — JWT registration & login	[]
3.3	Food endpoints	[]
3.4	Food entry & logging endpoints	[]
3.5	Daily summary & macro goal endpoints	[]
3.6	Middleware & background tasks	[]
Phase 4 — AI integration		upcoming
Deliverable: AI-powered food lookup and meal suggestions		Tools: Anthropic API · prompt engineering · structured outputs
4.1	Anthropic API setup & prompt engineering	[]
4.2	AI food lookup — name + weight to macros	[]
4.3	Natural language meal logging	[]
4.4	AI daily summary & suggestions	[]
Phase 5 — Flask admin & background jobs		upcoming
Deliverable: Admin dashboard + scheduled reports		Tools: Flask · Celery · Redis · Jinja2
5.1	Flask admin dashboard	[]
5.2	Celery & Redis — background jobs	[]
5.3	Weekly reports & email digests	[]
5.4	CSV & PDF export	[]
Phase 6 — React frontend		upcoming
Deliverable: Full UI accessible in the browser		Tools: React · Recharts · Tailwind · hooks
6.1	React setup & auth flow	[]
6.2	Dashboard — macro rings & daily summary	[]
6.3	Food log interface	[]
6.4	Weight & history charts	[]
6.5	Goal setting & AI suggestions UI	[]

Phase 7 — Testing & hardening		upcoming
Deliverable: Full test suite covering all endpoints		Tools: pytest · fixtures · mocking · coverage
7.1	pytest setup & fixtures	[]
7.2	Unit tests — models & utils	[]
7.3	Integration tests — API endpoints	[]
7.4	Mocking AI responses	[]
7.5	Global error handlers & input validation	[]
Phase 8 — Deployment & DevOps		upcoming
Deliverable: Live app deployed on the internet		Tools: Docker · Railway/Render · CI/CD · HTTPS
8.1	Docker & docker-compose	[]
8.2	CI/CD pipeline	[]
8.3	Railway/Render deployment	[]
8.4	Custom domain, HTTPS & monitoring	[]

Compounding concepts — used in every phase from introduction onward

Concept	How it compounds
Decorators	retry, rate_limit, auth, logging — wraps every function from Phase 1 onward
Type hints	annotates every function, feeds Pydantic and FastAPI from Phase 1 onward
Error handling	custom exceptions map to HTTP status codes from Phase 1 onward
Closures	captured state in decorators and factory functions
Generators	lazy data pipelines — CSV seeding, DB streaming, export
OOP & dataclasses	domain models promoted to ORM and Pydantic from Phase 2 onward
SQL & ORM	SQLAlchemy queries in every endpoint from Phase 2 onward
Async/await	FastAPI is async-native — every endpoint from Phase 3 onward
Pydantic validation	request/response schemas on every API boundary
Logging	structured logs across every module from Phase 1 onward
Caching	lru_cache to Redis — AI responses cached from Phase 4 onward
Testing	pytest fixtures and mocks applied retroactively to all phases in Phase 7